

EXERCISE: NEURAL NETWORKS EXTG11/EXTG15 Filip Winzell



Download the files in ANN.zip from the course web page in Canvas under “Exercise Neural Networks” and save them in your local directory.

You will need to install NumPy and Matplotlib to run it. For the first part of this exercise, it is recommended that you have an editor with a debug mode, for example [VS Code](#).

For the latter part, you will need to install PyTorch and networkx.

PyTorch:

Enter your operating system and preferences on this website and find out how to install:

<https://pytorch.org/get-started/locally/>

Note that you need at least Python 3.10 for the latest version of PyTorch

networkx:

Can be installed by simply entering the following into a terminal

➤ `pip install networkx`

Additional instructions can be found at: <https://pypi.org/project/networkx/>

If you have any issues with this, you can ask me at the exercise, message me on canvas or email at filip.winzell@math.lth.se

*****Submission*****

Write a nice, **individual** report according to the template (you can find it on Canvas under “Allmänt”).

Submit written report as a **pdf** document on Canvas within a week (2nd of April).

Note that everyone should submit their own report written in their own words. You are allowed to work in pairs on the exercise but we will not accept exact copies of reports.

Questions that you should answer are numbered **Q1 to Q12**.

Start your editor and open the `Perceptron.py` file. Look at the code in the method `perceptron()` : and insert a breakpoint before the while loop (line 41). Step through the code and make sure you understand it line by line.

Part 1

Q1. Which is the training task (logical operator) that the perceptron is trained on?

Q2. How do we measure its success (i.e. what is stored in the “SaveOutput() array)?

Run the code from start. How many generations did the perceptron need to find the solution?

Q3. Run the code 10 times, what is the average generation time for solving this task?

Change the code so that the perceptron is trained on the AND operator.

Q4. Run the code 10 times, what is the average generation time?

Q5. Which task takes longest to solve ”OR” or ”AND”? Why? The task is the same, to draw a straight line (hint: in what range are the weights first set?).

Q6. What learning rule is used?

Q7. Look up the NAND (Not AND) operator and write it down as a truth table. How does it compare to the AND operator?

Now train the perceptron on the XOR operator.

Q8. How good was the perceptron? What happens, why does the mean error reach 1 and why does it not stay at the minimal error value (0.5)? Hint: Check the stop condition (which is the “while” command).

Part 2

For this part you need to have PyTorch and networkx installed. See instructions for installation above.

Q9. Run `part2.py` and then look at the code (it calls the `Perceptron` object from `utils.py`)

What operator is the perceptron training on?

Train the network ten times. What was the average number of generations? Was it better or worse than `perceptron.py`?

If there was a difference, what explanation would you suggest?

Q10. Visualize the trained perceptron by running the `.visualize()` instance method from the `Perceptron` object. (e.g `perceptron.visualize()`)

Include the plotted network in your report.

Which activation function is used?

Q11. Now, change the target, `t`, in the prompt input so the perceptron trains on XOR. As the simple perceptron can only draw a straight line, you must change it to a network with multiple nodes that can draw two lines (a.k.a, a multi-layer perceptron). This cannot use the hard-limit

update rule as the perceptron but instead uses a loss function and backpropagation to update the weights.

Replace the perceptron from the previous exercise with a `MultiLayerPerceptron`. It takes a list of hidden node sizes as input, e.g. for an MLP with one hidden layer of 5 nodes, call:

```
mlp = MultiLayerPerceptron([5])
```

Train this on the previous task by calling the method: `mlp.train_mlp(..)` using the same arguments as for the previous exercise. (You might need to increase the number of epochs)

How does it perform?

Now decrease the number of neurons. How many neurons are needed in the hidden layer to solve the XOR problem?

You can visualize the mlp with `mlp.visualize()`. Include this plot on your final MLP in your report.

Q12. Add the following import to your code from the previous exercise

```
from utils import plot_db
```

Add to the end the following line (i.e after training):

```
plot_db(mlp, X, y)
```

Run the code again. What is plotted? Try adding more nodes and hidden layers. Explain what happens? Include at least two of these plots in your report